

Week 5. Cilium & eBPF

1. eBPF 기본 개념과 Hook

- eBPF: 커널 소스 수정이나 모듈 로드 없이, 커널 안에서 이벤트가 발생할 때마다 실행되는 작은 샌드박스 프로그램
- 네트워크 관점 → 패킷이 커널 경로의 특정 지점을 지날 때 내 코드를 끼워 넣는 것

로드부터 실행까지의 흐름

1. 제한된 C로 작성 → eBPF 바이트코드로 컴파일
2. 커널 로드 시 verifier가 정적 분석 (반드시 종료되는지, 잘못된 메모리 접근이 없는지)
3. 통과하면 JIT가 네이티브 기계어로 변환
4. hook에 부착 → 이벤트가 올 때마다 실행

구성요소	역할	Cilium에서 보이는 예
Program	hook에서 실행되는 코드	패킷 정책 판정, 서비스 로드밸런싱
Hook	프로그램이 붙는 커널 이벤트 지점	XDP, tc, cgroup
Map	커널 안 key-value 저장소. 프로그램과 유저스페이스(agent)가 공유	서비스/백엔드 맵, 정책 맵, <code>cilium_lb4_reverse_sk</code>
Verifier	로드 시 안전성 검증. 실패하면 로드 거부	커널 크래시를 막는 안전장치
JIT	바이트코드를 네이티브 코드로 변환	커널 코드에 가까운 실행 속도

Cilium이 사용하는 네트워크 Hook 위치 (패킷 경로 → 소켓/syscall 계층)

구분	Hook	위치	Cilium의 용도
패킷 경로	XDP	NIC 드라이버가 패킷을 받자마자, 다른 어떤 처리보다 앞	Prefilter(CIDR 기반 조기 drop), NodePort/LB 가속 (옵션)
패킷 경로	tc ingress/egress	인터페이스에서 스택의 초기 처리 후, L3 처리 전	Pod veth의 호스트 쪽에서 L3/L4 정책 집행, 엔드포인트로 리다이렉트, 서비스 LB
소켓(syscall) 계층	cgroup connect/sendmsg/recvmmsg	애플리케이션의 syscall 시점	socket-LB: ClusterIP를

구분	Hook	위치	Cilium의 용도
			connect() 시점에 백엔드 IP로 변환

- cgroup hook은 “패킷 수신 순서의 다음 단계”가 아니라 애플리케이션이 syscall을 부르는 시점에 실행됨 → Pod가 Service로 보낼 때는 패킷이 만들어지기 전에 가장 먼저 동작
- sockops 기반 소켓 가속(socket operations / socket send/recv)은 1.13에서 deprecated 된 과거 기능 → 현재 Service 처리에 쓰이는 socket-LB(cgroup hook)와는 별개

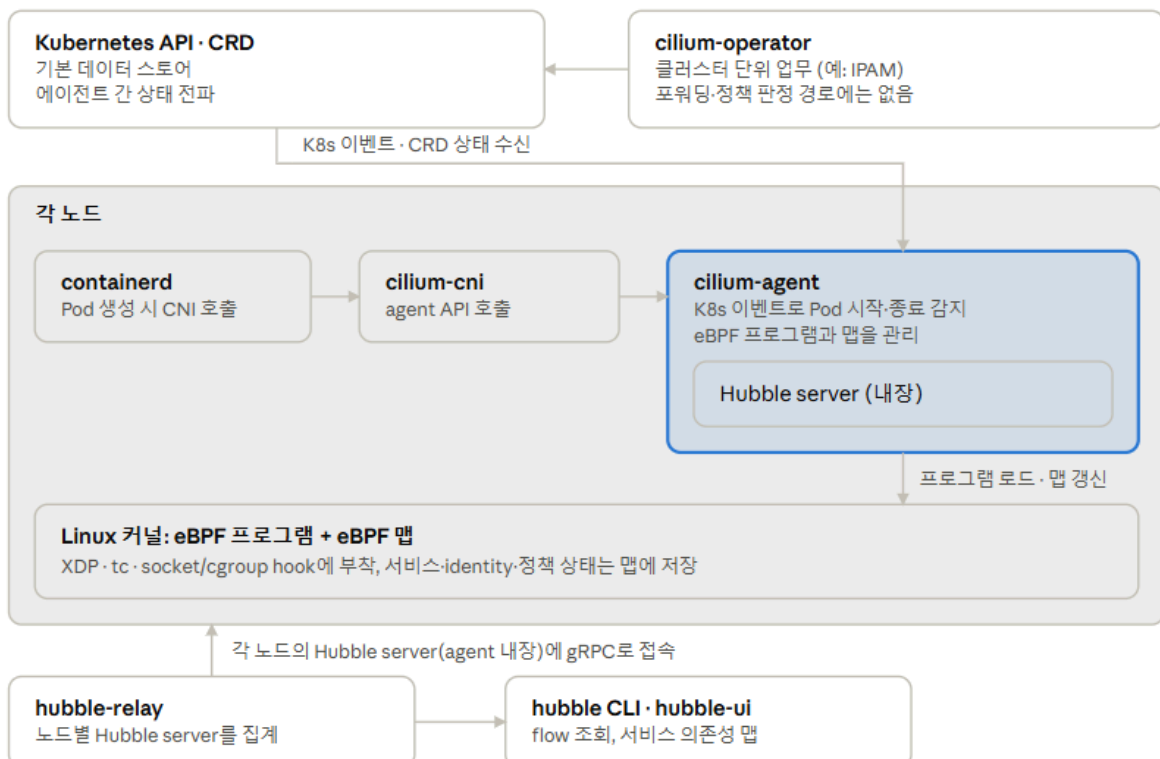
iptables vs eBPF

- iptables: 규칙을 커널에서 선형으로 처리
- eBPF: hook에서 맵을 조회해서 판정 → 처리 방식 자체가 다름

2. Cilium Architecture

- 구조: 노드마다 cilium-agent가 eBPF 프로그램과 맵을 관리
- CNI 플러그인 바이너리(cilium-cni): agent를 부르는 진입점

노드마다 cilium-agent가 eBPF를 관리하고, operator는 클러스터 단위 일만 맡는다



socket/cgroup hook=cgroup hook(socket-LB)

역할 정리

- **cilium-agent**: 각 노드에서 K8s 이벤트로 워크로드의 시작·종료 감지 → 네트워킹, 서비스 로드밸런싱, 정책, 모니터링 설정을 eBPF 프로그램과 맵으로 반영
- 관찰용 Hubble server도 agent 안에 내장
- **cilium-cni**: Pod 스케줄·종료 시 호출 → 노드의 Cilium API를 통해 데이터패스 설정 트리거 (실제 설정은 agent가 수행)
- **cilium-operator**: 클러스터에서 한 번만 하면 되는 일 담당
 - 포워딩·정책 판정 경로에는 없음 → 잠시 내려가도 클러스터는 대체로 동작
 - 구성에 따라 IPAM 지연, kvstore heartbeat 실패로 인한 agent 재시작 가능
- **데이터 스토어**: 기본은 Kubernetes CRD → 규모 최적화용으로 etcd 같은 key-value 스토어 선택 가능
- **CLI 두 종류**: **cilium-dbg** (노드 안, agent API와 eBPF 맵 확인) / **cilium** (원격에서 설치·점검) → 서로 다른 도구

3. Cilium Datapath

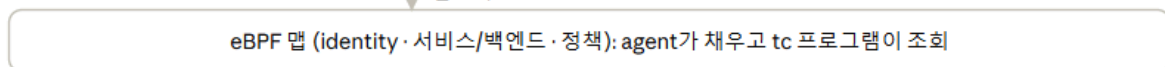
- 데이터패스의 중심 → Pod마다 있는 veth의 호스트 쪽 tc hook
- 정책 판정과 서비스 로드밸런싱이 주로 이 지점에서 일어남 (Pod에서 나가는 방향 기준)

Pod를 드나드는 패킷은 veth의 tc hook에서 정책과 서비스 판정을 거친다

나가는 방향 (Pod에서 밖으로)



맵 조회



들어오는 방향 (밖에서 Pod로)



강조 박스는 정책 판정과 서비스 LB가 일어나는 지점(tc hook)이다.

L7 Policy, L3 암호화, overlay는 옵션이고 컨 경우에만 경로에 들어온다.

들어오는 방향의 ingress 정책은 구성(예: endpoint routes)에 따라 veth 호스트 쪽 egress(to-container) 등 다른 지점에서 처리될 수 있음. 공통점은 모두 같은 eBPF 맵(identity·서비스·정책)을 조회해 판정한다는 것

설명

- Pod: veth pair로 호스트와 연결 → 호스트 쪽 tc ingress hook에 프로그램 부착 → 컨테이너를 나가는 모든 트래픽 감시·집행
- 들어오는 방향: NIC의 XDP prefilter가 맨 앞에서 불필요한 패킷 걸러냄(옵션) → eBPF 프로그램이 identity를 찾아 L3/L4 정책 판정 (실행되는 hook 위치는 구성에 따라 다름)
- L7 규칙이 걸린 트래픽만 노드의 Envoy 프록시로 전달 → 나머지는 커널 안에서 처리 완료
- 판정에 쓰는 정보는 eBPF 맵에 존재 → 맵은 agent가 채우고, hook의 프로그램은 조회만 수행
- 보이는 가상 인터페이스: `cilium_host`, `cilium_net` (오버레이 사용 시 `cilium_vxlan` 추가)

기존 방식과의 대비

- 기존 방식 → Service NAT(iptables), 노드 안 CNI 라우팅, 정책(iptables 또는 OVS)이 각각 다른 계층의 규칙
- Cilium → 세 단계가 같은 eBPF 프로그램 안(주로 veth 쪽 tc hook)에서 맵 조회로 이어짐
- 3주차 Calico → Pod별 `/32` 경로 + iptables 조합
- 한 줄 정리 → “규칙 체인을 도는 방식” vs “맵을 조회하는 방식”

4. kube-proxy Replacement

- 처리 대상: ClusterIP, NodePort, LoadBalancer, externalIPs, hostPort
- iptables 규칙 대신 eBPF 맵 조회로 처리 → kube-proxy 없이 클러스터 운영 가능

기존 방식과 비교

- kube-proxy는 진짜 프록시가 아니라 규칙을 관리하는 컨트롤러
- iptables 모드 흐름 → `KUBE-SVC-*` 체인이 확률(`statistic --mode random`)로 `KUBE-SEP-*`에 분기 → DNAT
- 서비스와 Pod가 자주 바뀌는 대규모 클러스터에서 NAT 테이블 부담이 커짐

- Cilium 같은 CNI가 이 로직을 다른 데이터 플레인으로 재구현함

```

2026-10-01 20:18.27 /home/mobaxterm kubectl get pods -n kube-system -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP
cilium-9v7gn                         1/1     Running   0           34m   17
2.18.0.7 cilium-lab-worker            <none>   <none>     0           34m   17
cilium-dc76n                         1/1     Running   0           34m   17
2.18.0.6 cilium-lab-control-plane       <none>   <none>     0           34m   17
cilium-envoy-csk4p                  1/1     Running   0           34m   17
2.18.0.6 cilium-lab-control-plane       <none>   <none>     0           34m   17
cilium-envoy-dpj79                  1/1     Running   0           34m   17
2.18.0.7 cilium-lab-worker            <none>   <none>     0           34m   17
cilium-operator-6d75bb9bb8-2z8cc    1/1     Running   0           34m   17
2.18.0.7 cilium-lab-worker            <none>   <none>     0           34m   17
cilium-operator-6d75bb9bb8-mswl5    1/1     Running   0           34m   17
2.18.0.6 cilium-lab-control-plane       <none>   <none>     0           34m   17
coredns-559f6c778d-g2btb           1/1     Running   0           35m   10
.0.0.130 cilium-lab-worker            <none>   <none>     0           35m   10
coredns-559f6c778d-mhp65           1/1     Running   0           35m   10
.0.0.150 cilium-lab-worker            <none>   <none>     0           35m   10
etcd-cilium-lab-control-plane        1/1     Running   0           36m   17
2.18.0.6 cilium-lab-control-plane       <none>   <none>     0           36m   17
hubble-relay-856fb9b979-g79kb       1/1     Running   0           34m   10
.0.0.95 cilium-lab-worker            <none>   <none>     0           34m   10
hubble-ui-6dcdfb579c-gmnfd          2/2     Running   0           34m   10
.0.0.252 cilium-lab-worker            <none>   <none>     0           34m   10
kube-apiserver-cilium-lab-control-plane 1/1     Running   0           36m   17
2.18.0.6 cilium-lab-control-plane       <none>   <none>     0           36m   17
kube-controller-manager-cilium-lab-control-plane 1/1     Running   0           36m   17
2.18.0.6 cilium-lab-control-plane       <none>   <none>     0           36m   17
kube-scheduler-cilium-lab-control-plane 1/1     Running   0           36m   17
2.18.0.6 cilium-lab-control-plane       <none>   <none>     0           36m   17

```

항목	kube-proxy (iptables)	Cilium kube-proxy replacement
규칙 위치	커널 NAT 테이블의 KUBE-SVC / KUBE-SEP 체인	eBPF 서비스·백엔드 맵
확인 명령	<code>iptables-save grep KUBE-SVC</code>	<code>cilium-dbg service list</code>
백엔드 선택	체인에서 확률 분기	기본 random, 옵션으로 Maglev 일관 해싱
Pod → ClusterIP (동서 트래픽)	패킷마다 DNAT	socket-LB: connect() 시점에 목적지를 백엔드 주소로 바꿈
외부 → NodePort (남북 트래픽)	노드 iptables에서 DNAT + SNAT	tc(또는 XDP 가속)에서 백엔드 선택, 원격 백엔드면 기본 SNAT, DSR 옵션
hostPort	portmap 같은 별도 플러그인	자체 지원 (portmap 불필요)

socket-LB가 핵심인 이유

- 앱은 ClusterIP에 연결했다고 인식
- 실제 커널 소켓은 이미 백엔드 Pod 주소에 연결 → 이후 패킷에 NAT 불필요
- 동작 지점 → cgroup의 connect / sendmsg / recvmsg hook

```
앱: connect(ClusterIP:80)
→ cgroup connect hook (eBPF)
→ 서비스 맵 조회 → 백엔드 Pod IP 선택
→ 소켓이 Pod IP:80에 직접 연결 (패킷 단위 DNAT 없음)
```

설치와 검증

```
# kube-proxy 없이 부트스트랩
kubeadm init --skip-phases=addon/kube-proxy

helm install cilium cilium/cilium --namespace kube-system \
  --set kubeProxyReplacement=true \
  --set k8sServiceHost=${API_SERVER_IP} \
  --set k8sServicePort=${API_SERVER_PORT}

# 검증
kubectl -n kube-system exec ds/cilium -- \
  cilium-dbg status | grep KubeProxyReplacement
kubectl -n kube-system exec ds/cilium -- \
  cilium-dbg service list
# 비어 있어야 정상 (신규 클러스터 기준)
iptables-save | grep KUBE-SVC
```

운영에서 자주 걸리는 지점

- `k8sServiceHost/Port` 필수 → kube-proxy가 없으면 `kubernetes` 서비스 ClusterIP를 처리해줄 주체가 없어서, agent가 API 서버 주소를 직접 알아야 함
- 서비스가 이미 도는 클러스터에서 kube-proxy를 끄거나 공존 상태를 바꾸면 기존 연결 끊김 → 두 NAT 테이블이 서로의 상태를 모름
- `kubeProxyReplacement=true`: 커널 지원이 없으면 agent가 기동 포기
- Helm 기본값 `false`: kube-proxy와 공존하는 모드 → Cilium은 클러스터 내부 ClusterIP의 패킷 단위 LB만 활성화, 나머지 서비스 처리는 kube-proxy가 담당
- 각 노드의 `InternalIP` 가 올바른 인터페이스에 붙어 있어야 함 (`kubelet --node-ip` 옵션 확인)
- Pod 종료 시 트래픽이 잠깐 끊길 수 있음 → SIGTERM과 EndpointSlice 갱신이 동시에 일어나기 때문. 권장: `preStop` sleep(수 초), 단 같은 grace period 안에서 소비되

므로 `terminationGracePeriodSeconds` 보다 짧게

가동 중인 클러스터에서 kube-proxy를 제거할 때

- DaemonSet만 지우면 노드에 iptables 규칙이 남음 → 정리 필요
- Cilium 설치 전까지 서비스 트래픽이 중단되고 기존 연결이 끊김

```
kubectl -n kube-system delete ds kube-proxy
# kubeadm 업그레이드 시 재설치 방지
kubectl -n kube-system delete cm kube-proxy
# 각 노드에서 root 권한으로: KUBE 규칙 제거
iptables-save | grep -v KUBE | iptables-restore
```

선택 옵션 요약

옵션	효과	주의
DSR (<code>loadBalancer.mode=dsr</code>)	백엔드가 클라이언트에 직접 응답, 클라이언트 소스 IP 보존	MTU 감소, AWS는 source/dest check 해제 필요
hybrid	TCP는 DSR, UDP는 SNAT	MTU 조정을 피하려는 절충안
Maglev (<code>loadBalancer.algorithm=maglev</code>)	백엔드 변경 시 재분배 최소화	서비스별 룩업 테이블로 메모리 증가, 남북 트래픽에만 적용
XDP 가속 (<code>loadBalancer.acceleration=native</code>)	드라이버 단에서 NodePort/LB 처리	드라이버의 native XDP 지원 필요

5. NetworkPolicy와 Identity

- 판정 기준: Pod IP X → 라벨에서 파생된 숫자 identity O
- Pod가 재생성되어 IP가 바뀌어도 정책 규칙은 그대로 유효
- NetworkPolicy는 CNI 스펙 밖 → Kubernetes는 리소스만 정의, 집행은 CNI 구현체의 몫
- 기존 방식 → Pod 단위로 네임스페이스 안에서 연결을 허용/차단하는 앱 중심 방화벽, 집행은 iptables나 OVS 같은 데이터 플레인
- Cilium → 같은 집행을 eBPF로 수행

Identity가 만들어지고 쓰이는 순서

1. Pod 생성 → agent가 해당 Pod를 endpoint로 등록 (endpoint = Cilium이 IP-identity를 붙여 관리하는 Pod 단위 객체)
2. endpoint의 security-relevant 라벨 조합에 클러스터 전역에서 유일한 숫자 identity 부여 (같은 라벨 조합이면 replica 수와 무관하게 identity 하나 공유)
3. IP → identity 매핑이 eBPF 맵에 저장
4. 패킷이 endpoint policy 프로그램에 도달 → 출발지 identity 조회 → (identity, 포트, 프로토콜) 규칙과 대조 → 허용 / 차단 / L7 프록시로 전달

항목	IP 기반 정책	Cilium identity 기반 정책
규칙의 기준	Pod IP 목록	라벨 → 숫자 identity
Pod 재생성으로 IP 변경	규칙 갱신이 필요	정책은 그대로, IP ↔ identity 매핑만 갱신
확장	Pod 수에 따라 규칙 증가	같은 라벨은 identity 하나를 공유

L3/L4는 커널 eBPF, L7은 Envoy

- L3/L4 → endpoint policy 오브젝트가 커널에서 직접 집행
- L7(HTTP 등) 규칙이 걸린 트래픽 → L7 Policy 오브젝트가 유저스페이스 Envoy로 리다이렉트
- 표준 NetworkPolicy는 L3/L4까지, L7 규칙은 `CiliumNetworkPolicy` (cilium.io/v2)로 작성

```

apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: frontend-to-api
spec:
  endpointSelector:
    matchLabels:
      app: api-server
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: frontend
      toPorts:
        - ports:
            - port: "8080"
      protocol: TCP

```

```
rules:
http:
-method: GET
path: "/api/v1/.*"

```

확인 명령

```
# endpoint와 identity
kubectl get ciliumendpoints -A
# identity ↔ 라벨
kubectl get ciliumidentity
kubectl -n kube-system exec ds/cilium -- \
    cilium-dbg policy selectors
hubble observe --verdict DROPPED --to-label app=api-server

```

운영 팁

- identity에 들어가는 라벨은 `app`, `role`, `env` 처럼 작고 일관된 집합으로 유지
- 값의 종류가 많거나 자주 바뀌는 라벨이 identity 계산에 들어가면 identity 수가 불어나 정책 관리가 어려워짐
- 3주차의 “Flannel 단독은 NetworkPolicy를 조용히 무시”와 대비 → Cilium은 정책을 직접 집행하고, 그 결과(DROPPED)를 Hubble이 그대로 보여줌

6. Hubble: 관찰과 정책이 연결되는 방식

- 관찰과 정책이 연결되는 이유 → 둘이 같은 eBPF 데이터패스 위에 있음
- 패킷을 판정하는 프로그램 = 이벤트를 내보내는 지점
- Hubble → 그 이벤트를 identity, 라벨, verdict와 함께 읽어서 표시

컴포넌트	위치	역할
Hubble server	각 노드, cilium-agent에 내장	노드의 eBPF 기반 가시성 데이터를 gRPC와 Prometheus 메트릭으로 제공
hubble-relay	독립 컴포넌트	모든 노드의 Hubble server에 접속해 클러스터 전체를 하나의 API로 제공
hubble CLI	클라이언트	relay 또는 로컬 server에서 flow 조회
hubble-ui	웹 UI	relay 기반 서비스 의존성·연결 맵

정책 운영 사이클과의 연결

1. 관찰: Hubble UI/CLI로 실제 서비스 간 통신(누가 누구에게, 어떤 포트로) 확인
2. 정책 작성: 확인한 통신을 기준으로 라벨 기반 정책 생성
3. 검증: `hubble observe --verdict DROPPED` 로 정책 때문에 끊긴 정상 트래픽이 없는지 확인
4. 보정: 차단된 flow의 출발지/목적지 라벨을 보고 규칙 수정
 - 표준 NetworkPolicy만 쓰는 환경 → 어떤 패킷이 정책에 걸렸는지 확인하기 어려움
 - Cilium → 판정 결과(FORWARDED/DROPPED)가 flow에 그대로 기록
 - 정책을 엄격하게 바꾸기 전에 audit 모드로 먼저 관찰하는 운용도 가능 → 트래픽을 실제로 막지 않고 정책에 걸렸을 트래픽만 기록 (agent 또는 endpoint 단위 설정, 주로 L3/L4)

Service 변환도 관찰된다

- socket-LB 변환 전후가 별도 이벤트로 기록
- `pre-xlate-fwd` → 서비스 주소로 나간 요청
- `post-xlate-fwd` → 백엔드 Pod 주소로 바뀐 시점
- `pre-xlate-rev` / `post-xlate-rev` → 응답 방향의 역변환 (백엔드 주소 → 서비스 주소)
- iptables 환경에서 conntrack이나 규칙 dump로 추적하던 일을 대체

L7 가시성은 비용이 있다

- L3/L4 flow → eBPF 이벤트로 확보
- L7(HTTP 등) → 트래픽이 Envoy 프록시를 거쳐야 보임
- L7 규칙(L7 CiliumNetworkPolicy)이 걸린 트래픽에서만 확인 가능 → 전체 트래픽에 무작정 켜지 않음
- 주의: L7 정책은 가시성과 함께 제한(집행)도 적용됨

활성화

```
cilium hubble enable --ui
```

자주 쓰는 조회

```
hubble observe --namespace default --last 100
```

```
hubble observe --verdict DROPPED --follow
```

```
hubble observe \
```

```
  --from-label app=frontend --to-label app=backend
```

```
hubble observe --protocol http --last 50
```

결론

- Cilium: CNI 표준 위에서 Pod 네트워킹, Service 로드밸런싱, 정책 집행, 관측을 모두 커널 안 eBPF 프로그램과 맵으로 처리하는 구현체

Q1. eBPF는 네트워크 처리의 어디에 들어가는가

- 패킷 경로 맨 앞(NIC 수신 직후, XDP) → 인터페이스(tc ingress/egress) → 애플리케이션이 syscall로 소켓을 다루는 지점(cgroup hook, socket-LB)까지 여러 지점에 부착
- Pod마다 있는 veth의 호스트 쪽 tc hook이 중심 → 여기서 정책 집행과 Service 로드밸런싱 수행 (Pod에서 나가는 방향 기준)
- 판정에 필요한 상태(서비스·백엔드, identity, 정책)는 eBPF 맵에 저장 → cilium-agent가 채움

Q2. Cilium이 기존 CNI와 다른 점

축	Flannel	Calico	Cilium
노드 안(①) 패킷 처리	표준 bridge 플러그인에 위임	iptables가 기본, nftables·eBPF 선택	eBPF (tc, XDP, cgroup hook)
노드 간(②) 연결	VXLAN 또는 host-gw	BGP, IPsec, VXLAN	터널(VXLAN/Geneve) 또는 native routing
Service 처리	kube-proxy	kube-proxy (eBPF 모드에서 대체 가능)	kube-proxy replacement (옵션)
정책 판정 기준	자체 구현 없음	라벨/selector 기반 + 자체 확장	라벨에서 파생한 숫자 identity

- ① 노드 안: iptables 대신 eBPF로 패킷 처리
- ② 노드 간: 오버레이, 라우팅 둘 다 지원

Q3. 트래픽 관찰과 정책 적용은 어떻게 연결되는가

- 정책을 판정하는 eBPF 프로그램 = 이벤트를 내보내는 지점 → 같은 데이터패스에서 집행과 관찰이 함께 발생
- 공통 키: identity → 정책은 identity로 작성, Hubble flow에는 출발지·목적지 identity, 라벨, verdict(FORWARDED/DROPPED) 기록
- 운영 사이클: 관찰 → 정책 작성 → `hubble observe --verdict DROPPED` 로 검증 → 보정

질문

1. 제일 헷갈렸던 게 socket-LB인데 앱이 connect()를 부르는 순간 목적지가 백엔드 Pod IP로 바뀐다고 이해함.
그러면 패킷 단계에서는 ClusterIP가 아예 안 보이는 건데 iptables 환경에서 했던 conntrack이나 tcpdump로 ClusterIP를 따라가는 이런 방식이 아니라 그냥 허블 이벤트로 확인하는 건지?
2. 애초에 가동 중인 클러스터에서 kube-proxy를 걷어내야 한다면 전환이 나은지 아님 새로 클러스터를 만들어서 옮기는 게 운영상 나을지?